



CO3-AT1-Debugging and Optimization

[← Back](#)

QUESTIONS

Q1ATM Withdrawal – Exception
Handling Debugging
10 marks**Q2**Hospital Registration –
NullPointerException
Debugging
10 marks**Q3**E-Commerce Product Search –
Array Exception Debugging
10 marks**Q4**Railway Reservation –
Synchronization Debugging
10 marks**Q5**Bank Account – Race Condition
Debugging

Q5 Bank Account – Race Condition Debugging

10 marks

A bank account contains ₹10,000. Two ATM threads simultaneously attempt to withdraw ₹7,000 and ₹6,000.

The following program contains a race condition.

Task:

1. Identify the race condition.
2. Identify the critical section.
3. Explain why both threads may pass the balance check.
4. Modify the complete program using synchronized.
5. Ensure that the balance can never become negative.
6. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class BankAccount {  
  
    int balance = 10000;  
  
    void withdraw(int amount) {  
  
        if (balance >= amount) {
```

10 marks

5/5 answered

```
System.out.println(
Thread.currentThread().getName()
+ " withdrawing " + amount);

try {
Thread.sleep(100);
}
catch (InterruptedException e) {
System.out.println("Interrupted");
}

balance = balance - amount;

System.out.println(
Thread.currentThread().getName()
+ " withdrawal successful");
}
else {
System.out.println(
Thread.currentThread().getName()
+ " insufficient balance");
}
}
}

class ATM extends Thread {

BankAccount account;
int amount;
```

```
ATM(BankAccount account, int amount) {  
    this.account = account;  
    this.amount = amount;  
}  
  
public void run() {  
    account.withdraw(amount);  
}  
}  
  
class BankApplication {  
  
    public static void main(String[] args)  
        throws InterruptedException {  
  
        BankAccount account = new BankAccount();  
  
        ATM atm1 = new ATM(account, 7000);  
        ATM atm2 = new ATM(account, 6000);  
  
        atm1.setName("ATM-1");  
        atm2.setName("ATM-2");  
  
        atm1.start();  
        atm2.start();  
  
        atm1.join();  
        atm2.join();  
  
        System.out.println(  
            "Final Balance = " + account.balance);  
        }  
    }  
}
```

Correct the program so that only one withdrawal succeeds.

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```

1 class BankAccount{
2     int balance = 10000;
3     synchronized void withdraw(int amount){
4         if(balance >= amount){
5             System.out.println(Thread.currentThread
6                 .getName() + " is withdrawing " + amount);
7             Thread.sleep(100);
8             }catch(Exception e){}
9             balance -= amount;
10            System.out.println("Withdrawal successful");
11        }else{
12            System.out.println("Insufficient balance");
13        }
14    }
15    public static void main(String[] args)
16        throws Exception{
17        BankAccount a = new BankAccount();
18        ATM t1 = new ATM(a, 7000);
19        ATM t2 = new ATM(a, 6000);
20        t1.setName("ATM-1");
21        t2.setName("ATM-2");
22        t1.start();
23        t2.start();
24        t1.join();

```

Test Input

stdin input (optional)

Output

ATM-1Withdrawing7000
 Withdrawal successful
 Insufficient balance
 Final Balance = 3000

 Pending manual review

```
25         t2.join();
26         System.out.println("Final Balance = "+a.balance);
27     }
28 }
29 class ATM extends Thread{
30     BankAccount a;
31     int amount;
32     ATM(BankAccount a,int amount){
33         this.a = a;
34         this.amount = amount;
35     }
36     public void run(){
37         a.withdraw(amount);
38     }
39 }
```



✓ Submitted